Atividade: Desenvolvendo um APP de mensagens

Navegação e Organização em Camadas

Msc. Nator Junior Carvalho da Costa

- Atividade pode ser realizada em grupos de até 3 pessoas;
- A entrega deve ser realizada pelo classrom, até a data limite;

1 Objetivo

Desenvolver um aplicativo simples de mensagens, inspirado no WhatsApp, utilizando React Native e Expo. A atividade tem como foco a navegacao entre telas, passagem de parametros, componentes reutilizaveis e organizacao em camadas (*Service* e *Repository*).

2 Requisitos

2.1 Tela Inicial (HomeScreen)

Desenvolva uma tela que apresente pelo menos quatro contatos ficticios, contendo:

- Nome do contato;
- Opcao para abrir a conversa.

Ao selecionar um contato, o aplicativo devera navegar para a tela de conversa, enviando seu identificador e nome como parametros.

2.2 Tela de Conversa (ChatScreen)

A tela devera apresentar:

- Nome do contato selecionado;
- Lista de mensagens;
- Mensagens enviadas alinhadas a direita;
- Mensagens recebidas alinhadas a esquerda;
- Campo de texto e botao para enviar.

O usuario devera conseguir digitar e enviar mensagens. Apos o envio, a mensagem devera aparecer na conversa e o campo sera limpo. Mensagens vazias nao devem ser permitidas.

2.3 Componente Mensagem

Crie um componente reutilizavel chamado `Mensagem`, que receba:

- `mensagem`: texto da mensagem;
- `recebida`: valor booleano que determina seu alinhamento e estilo.

3 Organizacao em Camadas

O aplicativo devera separar a interface, as regras de negocio e o armazenamento.

3.1 MensagemRepository

Classe responsavel pelo armazenamento das mensagens em um array de objetos.

Cada mensagem devera possuir os atributos: `id`, `contatoId`, `texto` e `recebida`.

Metodos obrigatorios:

- `getMensagens(contatoId)`: retorna as mensagens do contato;
- `adicionarMensagem(mensagem)`: armazena uma nova mensagem.

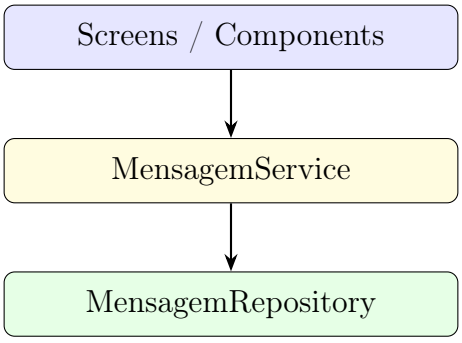
3.2 MensagemService

Classe responsavel pelas regras de negocio e pela comunicacao com o Repository.

Metodos obrigatorios:

- `getMensagens(contatoId)`: consulta as mensagens;
- `enviarMensagem(contatoId, texto)`: valida, cria e armazena uma mensagem.

A interface devera acessar os dados por meio do Service, sem acessar diretamente o Repository.



4 Estrutura do Projeto

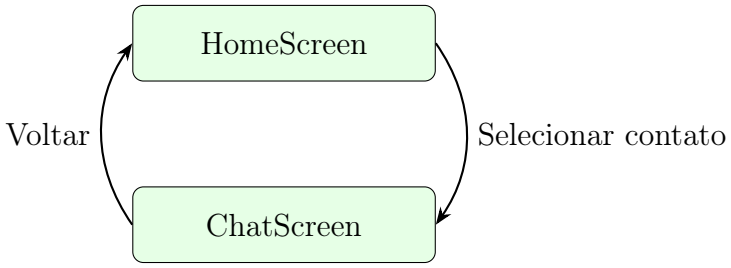
`src/`

```
  screens/
    HomeScreen.js
    ChatScreen.js
  components/
    Mensagem.js
  services/
    MensagemService.js
  repositories/
    MensagemRepository.js
App.js
```

5 Navegacao

Utilize o React Navigation com Stack Navigator.

O aplicativo devera permitir navegar da `HomeScreen` para a `ChatScreen`, passando os dados do contato selecionado, e retornar para a tela anterior.



6 Instrucoes

1. Crie o projeto utilizando Expo.
2. Configure o Stack Navigator.
3. Implemente a lista de contatos.
4. Desenvolva a tela de conversa.
5. Utilize `navigation.navigate()` e `route.params`.
6. Crie o componente `Mensagem`.
7. Implemente as classes `MensagemService` e `MensagemRepository`.
8. Utilize `useState` para atualizar as mensagens exibidas.
9. Teste o envio local e a navegacao.

7 Observacoes

- Nao sera utilizado MQTT, API ou banco de dados.
- Os dados serao armazenados em memoria.
- Utilize uma unica instancia do Service para compartilhar as mensagens entre telas.
- Cada conversa devera exibir apenas as mensagens do contato selecionado.

- Nao e necessario manter as mensagens apos reiniciar o aplicativo.